

# DISEÑO Y CONSTRUCCIÓN DE COMPILADORES

## Informe de Práctico de Máquina, 1ª Entrega

---



### Cátedra

- Aragón Victoria Soledad
- María José Garcarena Ucelay

### Alumnos

- Gerardo Ragazzi
- Cristian Ozan
- Roque Santa

# METODOLOGÍA DE TRABAJO

La dinámica de desarrollo del compilador se estructuró sobre un modelo colaborativo compuesto por un equipo de tres integrantes, apoyado en un ecosistema de herramientas modernas para el control de versiones y la escritura de código. Para garantizar el avance continuo y minimizar los conflictos en la integración, el ciclo de vida del proyecto se gestionó bajo los siguientes pilares:

- **Planificación y Asignación vía Roadmap:** La división técnica y operativa se realizó mediante un Roadmap general. Este documento actuó como la columna vertebral del proyecto, desglosando los requerimientos de la cátedra en tickets o hitos específicos, lo que permitió asignar responsabilidades claras a cada miembro del equipo y monitorear el progreso global. El trabajo no se repartió por módulo ni por persona, sino que se gestionó como una cola de tickets pequeños de la cual los miembros se auto-asignaban.
- **Gestión de Tickets y Desarrollo Ágil:** Se establecieron reglas estrictas para la gestión del desarrollo. Cada ticket correspondió a un *issue*, una rama dedicada (*feat/<id>* o *fix/<id>*), y un *Pull Request* (PR). Para garantizar el avance, se estableció una regla de caducidad: si un ticket permanecía 72 horas sin un PR abierto, retornaba a la cola automáticamente. La condición de finalización (*Definition of done*) requería obligatoriamente que el CI (Integración Continua) estuviera en verde.
- **Gestión Centralizada y Ramificación Estratégica en GitHub:** Todo el código fuente y las iteraciones del proyecto se centralizaron en un repositorio de GitHub. La arquitectura de ramas se diseñó en tres capas:
  - **Ramas de trabajo (*feat/<id>* / *fix/<id>*):** Ramas cortas para el trabajo local, separadas por función del *Parser Descendente Recursivo* (PDR).
  - **develop:** Acumuló el trabajo de la etapa en curso, recibiendo los PR validados de las ramas de trabajo.
  - **main:** Destinada exclusivamente a las entregas finales, requiriendo un PR desde develop con la aprobación formal de al menos dos integrantes (excluyendo al autor).
  - Tanto main como develop se configuraron como ramas protegidas, exigiendo revisión de pares, resolución de conversaciones y aprobación del CI sin excepciones.
- **Integración Continua y Validación de Pruebas:** Se implementó un sistema de pruebas automatizadas separado en dos categorías:
  - *validos/\*.c:* El CI validaba que no apareciera ninguna línea con Error N en la salida estándar.
  - *invalidos/\*.c:* Se realizaron afirmaciones estrictas contra archivos *.esperado* que contenían la secuencia exacta de errores a reportar, ejecutándose con un límite de tiempo (*timeout*) para detectar posibles bloqueos en la recuperación antipánico.

- **Trabajo en Paralelo y Mitigación de Conflictos:** El Roadmap permitió desacoplar el desarrollo. Para evitar conflictos en el archivo principal (parser.c), se prohibió reformatear o reordenar funciones, asegurando que los tickets de instrumentación modificaran bloques de código disjuntos. Además, se estableció una separación de propiedad por archivo, asignando los tickets de conjuntos FIRST1 exclusivamente al header de conjuntos, y los de instrumentación únicamente a parser.c.
- **Gramática de Referencia:** Se determinó que la gramática Extendida (BNFE) sería la fuente de verdad del proyecto, utilizando la BNF clásica únicamente como soporte para el cálculo de conjuntos de continuación (<resto X>) requeridos en los bucles de iteración.

Como resultado directo de este trabajo colaborativo e iterativo, se lograron estandarizar las reglas para el manejo de errores sintácticos. El fruto de estas discusiones y pruebas cruzadas se materializa en el siguiente marco de diseño para la recuperación antipánico.

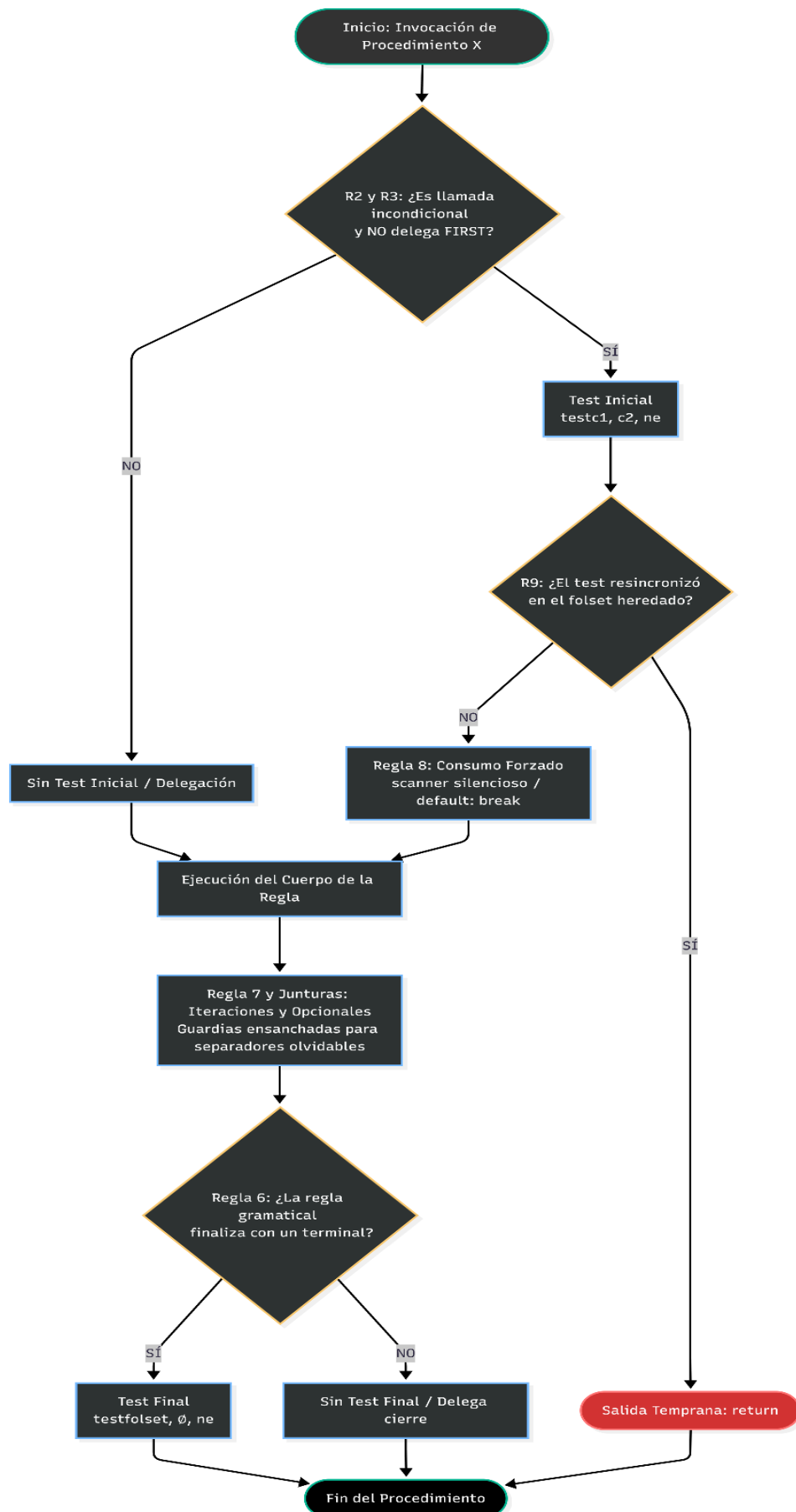
La necesidad de formalizar la instrumentación a través de un conjunto estricto de nueve reglas operativas surge de una limitación práctica del marco teórico base. Si bien la cátedra provee la firma de la función de recuperación antipánico y el esqueleto de los procedimientos reconocedores, decidir en qué línea exacta colocar cada protección, qué elementos incluir en los conjuntos de sincronización o cuándo delegar esta responsabilidad no es un proceso mecánico ni trivial. Cada una de estas decisiones arquitectónicas altera drásticamente el comportamiento del analizador sintáctico, determinando cuántos tokens se descartan ante un fallo y cuántos mensajes de error se emiten en la salida. Sin un criterio unificado y determinista guiado por la pregunta fundacional de cuánto confía un procedimiento en su llamador, la instrumentación se vuelve subjetiva y frágil. Una implementación arbitraria deriva rápidamente en fallos silenciosos, pérdida injustificada de código fuente válido, o en el temido efecto de "errores en cascada", donde un único defecto sintáctico desencadena múltiples reportes redundantes al desarmarse la pila de llamadas. Por lo tanto, la definición de estas reglas tuvo como propósito fundamental traducir la teoría abstracta de compiladores en un estándar de diseño predecible, cohesivo y medible, garantizando que el compilador resincronice el análisis con la mayor precisión posible y reporte cada falla exactamente una vez. En el desarrollo de las reglas a continuación se incluyen algunos ejemplos utilizando ciertos procedimientos de la gramática proporcionada por la cátedra.

## 1. REGLAS DEL ESQUEMA DE RECUPERACIÓN

El esqueleto de la cátedra provee **test(c1, c2, ne)** y los procedimientos reconocedores sin instrumentar. Decidir dónde va cada test, qué lleva cada conjunto y qué código de error corresponde no es mecánico: cada decisión cambia cuántos errores se reportan y cuánta entrada se descarta. Lo que sigue son las nueve reglas con las que se instrumentó el parser, cada una con una cadena del lenguaje que la ejerce y, donde fue posible medirlo, qué pasa si la regla no se aplica.

Las mediciones se obtuvieron compilando una variante del parser sin la pieza correspondiente y corriendo la misma entrada. La columna "sin la regla" no es una hipótesis: es la salida real de ese binario.

A continuación se deja un gráfico explicando la disposición de las reglas creadas durante el desarrollo;



## Regla 1 — El guardián no reemplaza al test

Un `if(lookahead_in(F_X)) X();` no reporta ni resincroniza: solo saltea. Es legítimo únicamente donde la BNFE dice `[ ]` o `{ }`, es decir donde la ausencia del elemento es legal. Pero un guardián legítimo tampoco es recuperación: si lo que sigue al opcional es un terminal obligatorio y el lookahead no es ni una cosa ni la otra, nadie reporta hasta el **match**, que además **no consume cuando falla**.

```
/* proposicion_compuesta */
```

```
if(lookahead_in(F_LISTA_DECLARACIONES))
```

```
    lista_declaraciones(folset | F_LISTA_PROPOSICIONES | CLLA_CIE);
```

```
if(lookahead_in(F_LISTA_PROPOSICIONES))
```

```
    lista_proposiciones(folset | CLLA_CIE);
```

```
match(CLLA_CIE, 25);
```

Cadena: void main() { \* a = 5; }

|                        | errores                   |
|------------------------|---------------------------|
| con el test de juntura | 52                        |
| sin él                 | 25 ("falta }", falso), 50 |

Con el símbolo intruso en el bloque más interno de cinco bloques anidados la diferencia se abre: **un error contra siete** (25, 50, 25, 25, 25, 25, 51), porque el `match(CLLA_CIE, 25)` cierra el bloque donde no corresponde y el desfasaje de llaves suma un reporte por cada nivel al desarmarse la pila.

## Regla 2 — Test inicial: cuándo va y qué lleva c1

Un procedimiento lleva test inicial cuando se lo invoca al menos una vez sin guardián y su primera sentencia no es una invocación a otro procedimiento. El `c1` depende de si el no terminal es anulable:

- $X \rightarrow \lambda \rightarrow c1 = \text{FIRST}(X)$
- $X \rightarrow \lambda \rightarrow c1 = \text{FIRST}(X) \cup \text{folset}$

En el segundo caso el test deja de preguntar "*¿empieza X acá?*" y pasa a preguntar "*¿estoy parado en un lugar del que este procedimiento puede seguir?*".

`/* declarador_init — anulable: <declarador init> ::= λ | = <constante> | [ ... ] */`

`test(F_DECLARADOR_INIT | folset, NADA, 47);`

Cadena: **int a;** — un programa **sin errores**.

|                         | errores                     |
|-------------------------|-----------------------------|
| con c1 = FIRST U folset | ninguno                     |
| con c1 = FIRST a secas  | 47 sobre un programa válido |

Es el caso que exige la consigna 11: en programas sin errores el compilador no debe marcar errores. Los dos anulables de esta gramática **<unidad traducción>** y **<declarador init>** se invocan sin guardián, así que ambos llevan test inicial y ambos entran por la segunda cláusula.

### Regla 3 — Delegación: sólo si los FIRST son iguales

Un procedimiento que arranca llamando a otro puede delegar su test inicial, pero solo si  $\text{FIRST}(\text{delegado}) = \text{FIRST}(\text{delegante})$ .

- `<expresión> ::= <expresión simple> <resto expresión>` — los FIRST coinciden, expresion **no lleva test propio** y delega en `expresion_simple`.
- `<expresión simple> ::= <operador opcional> <término> ...` —  $\text{FIRST}(\text{expresión simple}) \supseteq \text{FIRST}(\text{término})$  por el `[ + | - ]`, así que **no delega** y carga su test.

Corolario: si el delegado es anulable no hay ningún símbolo obligatorio en su entrada, y el delegante no puede descargar su test en él. **declaracion\_variable** no puede delegar en **declarador\_init**.

### Regla 4 — c1 y c2 son cosas distintas

- **c1** = qué debería haber acá. Es lo único que decide si se **reporta**.
- **c2** = dónde además es seguro frenar. Solo acota **cuánto se saltea**.

Frenar en un símbolo no es aceptarlo: el error ya se reportó, y c2 solo evita tirar más entrada de la necesaria. La consecuencia práctica es que **el folset heredado va en c2, nunca en c1**: si entrara en **c1**, el test dejaría de reportar exactamente en el caso que se quiere detectar.

## Regla 5 — c2 del test inicial

**c2 = folset heredado  $\cup$  puntos de recuperación internos del propio procedimiento.**

Un punto interno es una posición del cuerpo donde se puede retomar aceptando que se perdió un prefijo, y es admisible solo si al frenar ahí **el prefijo perdido queda identificado sin ambigüedad.**

- **Sí:** la cola de la propia producción. En <declaración parámetro> ::= <especificador de tipo> [ & ] ident [ [ ] ], frenar en & o en ident significa inequívocamente que faltó el tipo. Cadena void f(& a): se reporta un solo error y el parámetro se recupera entero.
- **No:** los símbolos de cierre ), ], }. Un token es punto de reconfiguración de X solo si X es quien lo **consume**.

## Regla 6 — Test final y construcción del folset

**Test final = test(folset,  $\emptyset$ , ne)**, sin FIRST(X) en c2. La formulación clásica de Wirth mete FIRST(X) como red de seguridad porque ahí los follower sets se calculan groseramente; la consigna 8 pide folsets **precisos por call site**, y con folsets precisos la red sobra.

El folset **acumula**, y "lo que sigue en ese call site" se lee **amplio**: no el símbolo inmediatamente siguiente, sino el FIRST de toda la cola de la producción a partir de ese punto, más lo heredado del llamador.

$X ::= \text{if } E \text{ then } S \text{ else } S \text{ fi}$

$E(\text{folset} + \{\text{then, else, fi}\} + \text{FIRST}_1(S));$

$S(\text{folset} + \{\text{else, fi}\} + \text{FIRST}_1(S));$

$S(\text{folset} + \{\text{fi}\});$

E recibe **then**, pero también **else, fi y FIRST(S)**: todo lo que puede aparecer después de E si lo que hay en el medio falta. Aplicado a **<proposición iteración> ::= while ( <expresión> ) <proposición>**:

**expresion(folset | CPAR\_CIE | F\_PROPOSICION);**

Con la lectura estrecha (**folset | CPAR\_CIE**), sobre **while (a < b { ... }** el test final de expresion saltaría el bloque entero buscando un **)** que no está. Con la amplia frena en **{**, el test pasa en silencio y el error lo reporta **match(CPAR\_CIE, 21)** con el mensaje correcto.

**Nota metodológica.** Este contraejemplo **no se pudo reproducir** aislando el cambio: al mutar solo esa línea la salida sigue siendo 21, porque el folset que

proposicion\_iteracion recibe de proposicion ya trae F\_PROPOSICION por otra vía. La regla se sostiene por construcción, no por medición. Se deja registrado antes que presentar un número que no se verificó.

## Regla 7 — Iteraciones: el chequeo en dos posiciones (consigna 12)

La BNFE viene colapsada: donde la BNF tiene  $X ::= A \langle Xtail \rangle / \langle Xtail \rangle ::= \lambda \mid A \langle Xtail \rangle$ , el parser tiene  $A \{ A \}$ . Para que las dos formas se comporten igual hace falta el chequeo **test(FIRST(A) | folset,  $\emptyset$ , ne)** antes de entrar al bucle y al cerrar cada iteración.

Lo que la teoría no dice es *quién* lo ejecuta. Si el cuerpo termina invocando un procedimiento que ya lleva test final, y por la regla 6 se lo invoca con folset | FIRST(A), ese test **es** el chequeo pedido: mismo c1, mismo c2 =  $\emptyset$ , y es la última instrucción que corre antes de evaluar la condición del **while**. Escribirlo otra vez es redundante.

**Caso particular: separador olvidable.** Cuando la condición del  $\{ \}$  es un separador fácil de olvidar la , se ensancha el guardián:

```
/* lista_declaraciones_param */

while(lookahead_in(CCOMA | F_DECLARACION_PARAMETRO))

{

    if(lookahead_in(CCOMA))

        scanner();

    else

        error_handler(64);

    declaracion_parametro(folset | CCOMA | F_DECLARACION_PARAMETRO);

}
```

Cadena: **void f(int a int b) { a = b; }**

|                            | errores            |
|----------------------------|--------------------|
| con el guardián ensanchado | 64 ("falta ,")     |
| sin él                     | 21, 49, 47, 23, 25 |

Cinco errores contra uno, y ninguno de los cinco menciona la coma. Consecuencia sobre la regla 6: F\_DECLARACION\_PARAMETRO pasa a ser parte de lo que legítimamente sigue al cuerpo en ese call site, así que entra en **el folset** que se le pasa, no en c2.

## Regla 8 — Forzar entrada (consigna 10)

Cuando un test inicial ya verificó el primer símbolo, el cuerpo **no lo vuelve a verificar**:

```
/* proposicion_compuesta — el test inicial ya verificó CLLA_ABR */
```

```
test(F_PROPOSICION_COMPUESTA, folset | F_LISTA_DECLARACIONES |  
F_LISTA_PROPOSICIONES, 49);
```

...

```
if(lookahead_in(CLLA_ABR))
```

```
    scanner();    /* en vez de match(CLLA_ABR, 24) */
```

El test se queda con la responsabilidad de reportar; el cuerpo consume **condicional y silenciosamente**. Un scanner() incondicional se comería el símbolo cuando el test frenó en c2; un match reportaría dos veces lo mismo.

Cadena: void main() seguido de int x; } — falta la llave de apertura.

|                         | errores |
|-------------------------|---------|
| con if + scanner()      | 49      |
| con match(CLLA_ABR, 24) | 49, 24  |

El test inicial reporta 49 y resincroniza en int; el match mira ese mismo int y reporta 24. Dos errores por una sola llave.

**Corolario — los default: de los switch no reportan**, en los procedimientos que llevan test inicial. Después del test, el lookahead está en c1 U c2. Si está en c1, alguna rama matchea. Llegar al default significa que está en c2 \ c1, o sea que el test ya reportó. Queda **default: break;**. Dos excepciones: **proposicion\_e\_s** no lleva test inicial y conserva su **error\_handler(29)**; **declarador\_init** no tiene default y no se le agrega, porque caer fuera de todos los case es la derivación λ, no un error.

## Regla 9 — Salida temprana cuando el test frenó en el folset

Al salir de un test inicial el lookahead puede estar en cuatro estados. Tres piden la misma acción; el cuarto no.

| estado al salir del test                             | acción                                                     |
|------------------------------------------------------|------------------------------------------------------------|
| no hubo error $\rightarrow$ lookahead $\in$ FIRST(X) | correr el cuerpo                                           |
| hubo error y resincronizó a algo de FIRST(X)         | correr el cuerpo: se perdió un prefijo, se retoma X entero |
| hubo error y resincronizó a un punto interno         | correr el cuerpo: lo previo se saltea solo (regla 8)       |
| hubo error y resincronizó al folset heredado         | <b>retornar</b> : nada del cuerpo aplica                   |

Como los tres primeros coinciden, el procedimiento no necesita saber si el test reportó: le alcanza con mirar el lookahead contra dos conjuntos que él mismo armó.

```
void X(set folset)
```

```
{  
  
    test(F_X, folset | INTERNOS_X, ne);  
  
    if(!lookahead_in(F_X | INTERNOS_X))  
  
        return;  
  
    /* cuerpo */  
  
}
```

```
Cadena: cout << ) ;
```

|                                   | errores    |
|-----------------------------------|------------|
| con la guarda en expresion_simple | 56         |
| sin ella                          | 56, 57, 23 |

Sin la guarda, expresion\_simple reporta parado en ), resincroniza hasta el ;, **el cuerpo sigue corriendo**, y el test de factor reporta un segundo error sobre ese mismo ;. La regla corta la cascada en el origen: expresion\_simple retorna y factor nunca llega a correr.

Cadena: `void main()` seguido de `}` — sin llave de apertura.

|                                               | errores    |
|-----------------------------------------------|------------|
| con la guarda en <b>proposicion_compuesta</b> | 49         |
| sin ella                                      | 49, 52, 25 |

Esta es la decisión de diseño propia del trabajo: la teoría no trae ningún mecanismo anticascada. Se evaluaron y descartaron tres alternativas —contador global de tokens con descarte en **error\_handler**, flag de pánico global, y que test devuelva dónde frenó— por razones de alcance y de estabilidad de los .esperado del CI.

## 2. CASOS EN DONDE EL ESQUEMA DE RECUPERACIÓN ANTIPÁNICO ES INSUFICIENTE Y EXPLICACIÓN DE RECONFIGURACIÓN.

Las nueve reglas fijan dónde va cada test y qué llevan c1 y c2, pero ninguna habla de la **juntura** entre un guardián y lo que viene después. Los lotes de prueba mostraron que esa juntura tiene una forma recurrente que el esquema no cubría.

### 2.1 El patrón guardián + terminal obligatorio

```
if(lookahead_in(FIRST_del_opcional))
```

```
    no_terminal_opcional(...);
```

```
match(TERMINAL_DE_CIERRE, ne);
```

Las dos piezas son correctas por separado. El guardián lo es por la regla 1 —la BNFE marca el elemento como [ ], su ausencia es legal— y el match lo es porque el terminal sí es obligatorio. El agujero es el hueco entre ambas: si el lookahead no está en FIRST(opcional) **ni** es el terminal de cierre, el guardián se saltea en silencio y el único que reporta es el match. De ahí salen dos consecuencias, y la segunda pesa más:

1. **El mensaje es falso.** No falta el terminal: sobra un símbolo antes de él.
2. **match no consume cuando falla.** El símbolo intruso sigue en el lookahead, la producción se da por terminada donde no corresponde, y el desfasaje se propaga hacia arriba por los tests finales de cada llamador. Esto es lo que convierte un error en cinco o siete.

Es un caso de la consigna 14: el problema no es el token, es que **nadie resincroniza antes del match**. La regla 9 corta la cascada dentro de un procedimiento que ya reportó; acá no había reporte alguno que la disparara.

**Reconfiguración (consigna 15).** Un test inmediatamente antes del guardián, con el conjunto director completo de la juntura en c1 y el folset heredado en c2:

```
test(FIRST_del_opcional | TERMINAL_DE_CIERRE, folset, ne);
```

El folset va en c2 y no en c1 por la regla 4: en c1 el test dejaría de reportar exactamente en el caso que se quiere cazar.

Los tres sitios de la gramática con esta forma:

| procedimiento             | opcional                                    | cierre | ne |
|---------------------------|---------------------------------------------|--------|----|
| definicion_funcion        | <lista decl<br>parámetros>                  | )      | 41 |
| proposicion_compu<br>esta | <lista declaración>,<br><lista proposición> | }      | 52 |
| llamada_funcion           | <lista expresiones>                         | )      | 56 |

Medición, errores reportados antes → después:

| cadena                                           | antes | después |
|--------------------------------------------------|-------|---------|
| void f(double dato)                              | 7     | 1       |
| void f(, int x)                                  | 6     | 1       |
| void f((int x)                                   | 8     | 1       |
| void f(& int x)                                  | 6     | 1       |
| { * a = 5; }                                     | 2     | 1       |
| * en el más interno de cinco<br>bloques anidados | 7     | 1       |
| fop4( , b)                                       | 5     | 2       |

Ningún caso válido cambia de salida: el test pasa en silencio cuando el lookahead está en c1.

**Elección de ne.** El 41 de definicion\_funcion y el 52 de proposicion\_compuesta son los mismos códigos que ya usan especificador\_tipo y proposicion en sus tests iniciales, así que la juntura habla el mismo idioma que el procedimiento al que iba a entrar. Para llamada\_funcion no hay código propio de <lista expresiones>; el 56 es prestado de expresion\_simple y describe mal lo que falta —sobre fop4( , b) lo que sobra es una coma, no lo que falta una expresión simple—. Se dejó el 56 porque es el que la juntura efectivamente hereda; los slots 36–39 están reservados para errores personalizados y son el lugar natural si se decide numerarlo aparte.

## 2.2 La regla 9 faltaba en proposicion\_compuesta

Agregar el test de juntura destapó una omisión anterior. **proposicion\_compuesta** lleva test inicial (el 49) y por lo tanto le corresponde la guarda de la regla 9, que no tenía. Sobre **void main()** seguido de **}**: el test inicial no encuentra **}** en **c1** ni en **c2**, reporta 49 y **descarta el }** al resincronizar; queda **CEOF**; el test de juntura corre sobre ese **CEOF** —que está en el folset heredado pero no en su **c1**— y reporta un 52 sobre un token ya en estado de error; el **match(CLLA\_CIE, 25)** agrega un tercero. Con la guarda: 49 solo. **Tres errores a uno**, mejor que antes de introducir el test de juntura, no solo mejor que sin guarda.

## 2.3 Lo que la reconfiguración no cubre

**Basura en el medio de una lista, no al principio.** **{ a = 1; \* a = 2; }** sigue reportando desde el test final de **proposicion\_expresion** (55) y no desde el inicial de **proposicion** (52). El test de la juntura ya pasó cuando eso ocurre. No es cascada —un símbolo, un error— sino el punto de detección más externo.

Vale la pena ver los dos errores de **a = 5 + ; b = \* 3;** juntos, porque la asimetría entre ellos no es un defecto:

- El **;** que sigue al **+** aparece **dentro** de la expresión simple, con su test inicial ya pasado (el 5 lo satisfizo), así que el flujo baja hasta **factor** y reporta 57.
- El **\*** que sigue al **=** aparece **al comienzo** de la expresión simple de la derecha, así que lo intercepta el test inicial de **expresion\_simple** y reporta 56; **factor** nunca corre, por la regla 9.

Mismo símbolo inesperado, distinto punto de detección según en qué lugar de la producción caiga. El mismo fenómeno explica que **int a;** seguido de **\* a = 5;** reporte 51 (test final de declaración) y no 52 (test inicial de proposición): el test de salida del no terminal anterior intercepta antes que el de entrada del siguiente. Mientras haya tests finales, ese es el diagnóstico que corresponde.

**Mensajes que mienten.** Tres casos del lote dicen "Falta X" con X presente en la línea:

**int a[ 3 \* ] ;** -> Error 22: Falta **]** (el **]** está; sobra el **\***)

**cin >> 5;** -> Error 23: Falta **;** (el **;** está; el 5 no es variable)

**a = 10 / b = 20;** -> Error 23: Falta **;**

Es el **match** reportando el terminal que esperaba cuando lo que hay es basura antes. Corregirlo requiere recuperación a nivel de frase —asumir el token faltante e insertarlo—, que es otra técnica y queda fuera del alcance de esta entrega.

**Un error espurio conocido.** `int a = x;` reporta dos errores: el 62 correcto (se esperaba constante) y un 64: **Falta** , que es invención. Sale del guardián ensanchado de **declaracion\_variable** (regla 7), que ve el `x` como si empezara otro declarador. Es el precio del tratamiento del separador olvidable: sin él, `int a = 1 b = 2;` deja de reportar la coma faltante. Se optó por conservar el guardián porque la omisión de la coma es el error que la consigna 12 pide tratar.

### 3. ALCANCE DEL ESQUEMA

El lote de prueba tiene **76 casos**: 11 programas válidos y 65 inválidos, cada uno con su **archivo .esperado** que fija la secuencia exacta de errores. El CI los corre en cada push y falla si alguna secuencia difiere.

#### 3.1 Un error por falta

| errores reportados | cantidad de tests |
|--------------------|-------------------|
| 1                  | 49                |
| 2                  | 10                |
| 3                  | 3                 |
| 4                  | 2                 |
| 5                  | 1                 |

**Los seis casos con tres o más errores no son cascada.** En todos, la cantidad de errores coincide con la cantidad de faltas reales en la entrada:

- **03\_falta\_parentesis\_cierre** reporta 5 y tiene 5 paréntesis omitidos, uno por cada producción de la BNFE que consume un `)` —definición de función, factor parentizado, invocación, selección e iteración—.
- **11\_cascada\_funcion\_completa** tiene tres faltas —un `)` en el `while`, un `+ *` en una expresión y un `;` en el `return`— y reporta exactamente tres, cada una con el mensaje correcto y en su línea.
- **14\_expresion\_simple\_signo\_sin\_operando** tiene cuatro líneas con error y reporta cuatro 57, uno por línea: no hay arrastre entre líneas.

Los 11 programas válidos se compilan **sin ningún error reportado**, que es la consigna 11.

#### 3.2 Cobertura verificada por mutación

La consigna 13 pide ver hasta dónde ayuda el esquema. Un lote que pasa no demuestra nada por sí solo: demuestra que el parser hace lo que el lote espera, no que el lote sea capaz de detectar una regresión. Para medirlo se hizo **análisis de mutación**: se enumeraron los 27 puntos de recuperación del parser —26 llamadas a `test` más la guarda de la regla 9—, se generó una variante del binario por cada uno eliminándolo, y se corrió el lote completo contra cada variante.

Un punto está **cubierto** si al quitarlo algún test falla. Si ninguno falla, esa pieza puede borrarse sin que el CI se entere.

|              |                             |
|--------------|-----------------------------|
|              | <b>puntos sin cobertura</b> |
| lote inicial | <b>14 de 27</b>             |
| lote final   | <b>0 de 27</b>              |

Los 14 huecos se cerraron con casos contruidos específicamente para cada uno. Tres de ellos costaron encontrarlos y dicen algo del diseño:

- El test inicial de **declarador\_init** (47) es **inalcanzable por el camino global**: cualquier símbolo intruso lo intercepta antes el **test(..., 43)** de **especificador\_declaracion**. Solo se llega a él por una declaración local, con **void main() { int a \* ; }.**
- El test final de **declarador\_init** (48) exige que el switch haya consumido algo primero, así que requiere un **[ ... ]** o un **= <constante>: int a[ 3 \* ] ;.**
- El test final de **factor** (58) no se alcanza con un factor que sea variable o constante, porque los tests de **variable** y **constante** reportan antes. Hace falta un factor parentizado o una constante string: **a = (b) ] ;.**

Que la cadena de tests se cubra a sí misma en esos tres puntos es evidencia de que la instrumentación está densa, no rala.

### 3.3 Contraste con un compilador de producción

Como referencia externa se corrieron los casos análogos por gcc, adaptando el símbolo intruso, en C el \* puede iniciar una sentencia (dereferencia) y en esta gramática no, así que se usó %.

| <b>caso</b>                                          | <b>gcc</b> | <b>ucc</b> |
|------------------------------------------------------|------------|------------|
| símbolo intruso al inicio de un bloque               | 1          | 1          |
| el mismo en el más interno de cinco bloques anidados | 1          | 1          |
| dos bloques hermanos, uno intruso en cada uno        | 2          | 2          |
| falta la } de cierre                                 | 1          | 1          |
| falta la { de apertura                               | 3          | 1          |

El caso de los bloques anidados es el más informativo: gcc **no** deja que el desfasaje de llaves suba un error por nivel, y ucc con el test de juntura tampoco. Es decir que la

reconfiguración de la sección 2 lleva al parser al mismo comportamiento que un compilador maduro en el caso donde el esquema base fallaba.

clang, sobre los mismos casos, reporta 2 en el caso anidado. Que dos compiladores de producción difieran entre sí confirma que no hay un óptimo único en recuperación de errores; lo que sí hay es un piso —un error por falta, sin arrastre— y ese piso se alcanza.

### 3.4 Conclusión

El esquema antipánico, instrumentado con las nueve reglas y reconfigurado en los tres puntos de la sección 2, alcanza:

- un error por falta en 49 de 65 casos inválidos, y en los 16 restantes la cantidad de errores coincide con la cantidad de faltas;
- cero errores en los 11 programas válidos;
- ninguna ejecución que aborte, se cuelgue o descarte el resto del archivo;
- los 27 puntos de recuperación protegidos por al menos un caso de prueba.

Lo que le falta para ser bueno en vez de correcto es la calidad de los mensajes: los tres "Falta X" de la sección 2.3 y el 64 espurio de `int a = x;` se resuelven con recuperación a nivel de frase, que es una técnica distinta y material de una etapa posterior.

## 4. RESUMEN DE PROCEDIMIENTOS IMPLEMENTADOS

A los fines de organizar la documentación técnica sin perder el rigor analítico, el sistema de rutinas se desglosa y analiza a continuación agrupado en sus tres regiones gramaticales fundamentales.

### Región Raíz y Declaraciones Globales (N7 a N11)

1. **<unidad de traducción> (unidad\_traducccion):**  
Reconoce la repetición global { <declaraciones> }. Al ser anulable (lambda), porta test inicial con folset: test(F\_UNIDAD\_TRADUCCION | folset, folset, 40). Su bucle while sale únicamente ante CEOF que valida el driver; no lleva test final propio.
2. **<declaraciones> (declaraciones):**  
Reconoce tipo, identificador y bifurcación. Delega test inicial en especificador\_tipo() (Regla 3), al cual le pasa el folset extendido folset | CIDENT | F\_ESPECIFICADOR\_DECLARACION. Consume el nombre con match(CIDENT, 17) y delega su cierre en especificador\_declaracion().
3. **<especificador de tipo> (especificador\_tipo):**  
Reconoce void, char, int, float. Porta test inicial test(F\_ESPECIFICADOR\_TIPO, folset, 41), cuatro ramas case con scanner(); break; listas para acciones semánticas y default: break; silencioso (Regla 8). Concluye con test final test(folset, NADA, 42).
4. **<especificador de declaración> (especificador\_declaracion):**  
Bifurca hacia función (CPAR\_ABR) o variable. Porta test inicial test(F\_ESPECIFICADOR\_DECLARACION, folset, 43), cuyo c\_1 incluye ; por ser anulables los declaradores (Guía 8). Su switch tiene default: break; silencioso (Regla 8) y delega el cierre en sus subordinados.
5. **<definición de función> (definicion\_funcion):**  
Reconoce cabecera y cuerpo ( [params] ) { ... }. No lleva test inicial (( garantizado). Consume delimitadores con match(CPAR\_ABR, 20) y match(CPAR\_CIE, 21). Pasa folset amplio a parámetros (folset | CPAR\_CIE | F\_PROPOSICION\_COMPUESTA) y delega el cierre en proposicion\_compuesta().
6. **<declaración de parámetro> (declaracion\_parametro):**  
Reconoce tipo [&] ident [()]. Delega test inicial en especificador\_tipo() con puntos internos folset | CAMPER | CIDENT (Reglas 3 y 5). Consume match(CIDENT, 17), corchetes opcionales con if(lookahead\_in(CCOR\_ABR)) y match(CCOR\_CIE, 22), y concluye con test final test(folset, NADA, 45).

7. **<lista declaración de parámetros> (lista\_declaraciones\_param):**

Reconoce parámetros formales separados por coma. Delega test inicial y final en `declaracion_parametro()`. Implementa bucle ensanchado `while(lookahead_in(CCOMA | F_DECLARACION_PARAMETRO))` con `match(CCOMA, 64)` para tratar la coma como separador olvidable (Regla 7).

8. **<declaración de variable> (declaracion\_variable):**

Reconoce <declarador init> [, <lista decl init>] ;. No lleva test inicial propio. Trata la coma opcional con guardián ensanchado `if(lookahead_in(CCOMA | F_LISTA_DECLARACIONES_INIT))` y `match(CCOMA, 64)`. Concluye con `match(CPYCOMA, 23)` y test final `test(folset, NADA, 51)`.

9. **<declarador init> (declarador\_init):**

Reconoce inicialización escalar (= cte), dimensión de arreglo ([cte]) o lambda. Al ser anulable, porta test inicial con folset: `test(F_DECLARADOR_INIT | folset, NADA, 47)`. Su switch carece de default por derivar en lambda (Reglas 1 y 8). Consume dimensión entera con `if+scanner()`, delimitadores con `match()`, y concluye con test final `test(folset, NADA, 48)`.

10. **<lista declaraciones init> (lista\_declaraciones\_init):**

Reconoce identificador y declarador repetidos. Valida identificadores con `match(CIDENT, 17)` e itera con guardián ensanchado `while(lookahead_in(CCOMA | F_LISTA_DECLARACIONES_INIT))` con `match(CCOMA, 64)` (Regla 7). Delega su cierre en la última variable.

11. **<lista de inicializadores> (lista\_inicializadores):**

Reconoce constantes dentro de arreglos { 1, 2, 3 }. Delega en `constante()`, iterando con guardián ensanchado `while(lookahead_in(CCOMA | F_CONSTANTE))` y `match(CCOMA, 64)` (Regla 7). No porta test en el bucle para no rechazar los seguidores de cierre (}) heredados en el folset.

12. **<declaración> (declaracion):**

Reconoce declaraciones locales dentro de un bloque. Delega test inicial en `especificador_tipo()` (Regla 3), delega la lista en `lista_declaraciones_init()` y concluye consumiendo el delimitador con `match(CPYCOMA, 23)` y test final `test(folset, NADA, 51)`.

13. **<lista de declaraciones> (lista\_declaraciones):**

Reconoce repetición de declaraciones { <declaración> }. Delega en `declaracion()`. Su bucle `while(lookahead_in(F_DECLARACION))` prescinde de tests internos explícitos puesto que el test final del subordinado garantiza el *follower set* en dos posiciones (Regla 7).

## Región de Proposiciones y Bloques (N4 a N6)

### 14. <proposición compuesta> (proposicion\_compuesta):

Reconoce bloques { [decl] [prop] }. Porta test inicial test(F\_PROPOSICION\_COMPUESTA, folset | F\_LISTA\_DECLARACIONES | F\_LISTA\_PROPOSICIONES, 49). Consume la llave con if+scanner() (Regla 8) y activa la guarda de salida temprana de la Regla 9. Concluye consumiendo el cierre con match(CLLA\_CIE, 25).

### 15. <lista de proposiciones> (lista\_proposiciones):

Reconoce repetición de sentencias en bloque. Delega test inicial en proposicion(). Su bucle while(lookahead\_in(F\_PROPOSICION)) absorbe el test del cuerpo en cada vuelta sin duplicación (Regla 7) y finaliza ante } o seguidores heredados.

### 16. <proposición> (proposicion):

Bifurca hacia las 6 clases de sentencias. Porta test inicial test(F\_PROPOSICION, folset, 52). Su switch(lookahead()) transfiere el control a cada función específica y posee default: break; silencioso (Regla 8). Delega su cierre en la alternativa elegida.

### 17. <proposición iteración> (proposicion\_iteracion):

Reconoce while ( expr ) prop. No lleva test inicial (CWHILE garantizado). Valida terminales con match(CWHILE, 27), match(CPAR\_ABR, 20) y match(CPAR\_CIE, 21). Pasa a expresión folset amplio folset | CPAR\_CIE | F\_PROPOSICION (Regla 6) y delega cierre en proposicion(folset).

### 18. <proposición selección> (proposicion\_seleccion):

Reconoce if ( expr ) prop [ else prop ]. No lleva test inicial (CIF garantizado). La expresión recibe folset amplio incluyendo F\_ELSE\_OPCIONAL. La rama then recibe folset | F\_ELSE\_OPCIONAL. La rama else opcional se evalúa con if(lookahead\_in(F\_ELSE\_OPCIONAL)) scanner(); proposicion(folset);

### 19. <proposición e/s> (proposicion\_e\_s):

Reconoce cin >> ... ; y cout << ... ;. No lleva test inicial (garantizado por llamador); su default emite error\_handler(29); break;. Emplea bucles ensanchados con match(CSHR, 30) y match(CSHL, 31) para redirecciones olvidadas (Regla 7). Cierra con match(CPYCOMA, 23) y test final test(folset, NADA, 53).

### 20. <proposición retorno> (proposicion\_retorno):

Reconoce return [expr] ;. No lleva test inicial; consume inmediatamente scanner(); al estar pre-validado CRETURN (Guía 7). Expresión opcional guardada por if(lookahead\_in(F\_EXPRESION)). Concluye con match(CPYCOMA, 23) y test final test(folset, NADA, 54).

21. **<proposición expresión> (proposicion\_expresion):**

Reconoce expresiones por efecto o sentencia nula [expr] ;. No lleva test inicial (invocada condicionalmente). Expresión opcional guardada por if(lookahead\_in(F\_EXPRESION)). Concluye consumiendo el delimitador con match(CPYCOMA, 23) y test final test(folset, NADA, 55).

## **Región de Expresiones y Operadores (N1 a N3)**

22. **<expresión> (expresion):**

Reconoce asignaciones (:=) u operaciones relacionales. Delega test inicial en expresion\_simple() (Regla 3). Su bucle while consume asignación o relacionales in-line con scanner() e invoca a expresion\_simple() con folset acumulado. Delega el test final en el subordinado.

23. **<relación> (In-line en expresion):**

Reconoce operadores relacionales (!=, ==, <, <=, >=, >). Carece de función independiente en C (Guía 6) por constituir una alternancia no recursiva de terminales que solo aparece dentro de <expresión>, procesándose in-line.

24. **<expresión simple> (expresion\_simple):**

Reconoce signo opcional, término y sumas/restas/disyunciones (+, -, ||). Porta test inicial test(F\_EXPRESION\_SIMPLE, folset | F\_RESTO\_EXPRESION\_SIMPLE, 56). Posee la guarda de 3 salidas (N1) para consumir || huérfanos sin descartar la expresión. Itera en while con scanner() y termino().

25. **<término> (termino):**

Reconoce productos, divisiones y conjunciones (\*, /, &&). Delega test inicial en factor() (Regla 3). Su bucle while(lookahead\_in(F\_RESTO\_TERMINO)) consume operadores con scanner() e invoca a factor() con folset amplio, sin tests redundantes en el bucle (Regla 7).

26. **<factor> (factor):**

Reconoce identificadores, constantes, subexpresiones ((expr)), negaciones !expr y cadenas. Porta test inicial test(F\_FACTOR, folset, 57) y switch con default: break; silencioso (Regla 8). Bifurca hacia función si lexema[0] == 'f' o variable. Concluye con test final test(folset, NADA, 58).

27. **<constante> (constante):**

Reconoce literales enteros, flotantes y caracteres. Porta test inicial test(F\_CONSTANTE, folset, 62). Conserva tres case independientes con scanner(); break; para futuros atributos semánticos, default: break; silencioso (Regla 8) y test final test(folset, NADA, 63).

28. **<variable> (variable):**

Reconoce identificadores escalares o indexados (ident[expr]). Porta test inicial test(F\_VARIABLE, folset | CCOR\_ABR, 59) con punto interno seguro en corchete [ (Regla 5). Consume identificador con if+scanner() (Regla 8), valida corchetes con match(CCOR\_CIE, 22) y concluye con test final test(folset, NADA, 60).

29. **<llamada a función> (llamada\_funcion):**

Reconoce subrutinas ident( [args] ). No lleva test inicial propio (nombre garantizado por llamador). Consume ( con scanner(), argumentos opcionales con if(lookahead\_in(F\_LISTA\_EXPRESIONES)) con folset amplio folset | CPAR\_CIE, y cierra con match(CPAR\_CIE, 21) y test final test(folset, NADA, 61).

30. **<lista de expresiones> (lista\_expresiones):**

Reconoce argumentos reales separados por coma. Delega en expresion(). Itera con guardián ensanchado while(lookahead\_in(CCOMA | F\_EXPRESION)) con match(CCOMA, 64) para comas olvidadas entre parámetros reales (Regla 7). Sin test final propio.

## 5. CIERRE FINAL

El presente informe sintetiza los lineamientos arquitectónicos, las reglas maestras de recuperación antipánico y las principales reconfiguraciones que gobernaron esta primera etapa. Para una profundización exhaustiva procedimiento por procedimiento, la justificación formal de cada conjunto seguidor (folset) por call site, las resoluciones de discrepancias entre BNF y BNFE, y el historial de discusión técnica de cada ticket, se remite al documento **\*\*entrega1.md\*\***, el cual actúa como la bitácora viva y fuente de verdad técnica de todo el desarrollo.

La culminación de esta primera etapa del compilador ucc nos permitió contrastar la teoría abstracta de diseño de compiladores con las complejidades reales de instrumentar un Parser Descendente Recursivo sobre un sublenguaje no trivial como C++.

A lo largo del desarrollo, los principales hitos y aprendizajes consolidados fueron:

1. **De la heurística al estándar determinista:** Comprobamos que la recuperación antipánico no puede depender de la intuición individual de cada programador. La formulación de las 9 reglas de diseño (Ticket T5) y la pregunta fundacional ("¿cuánto confía un procedimiento en su llamador?") transformaron decisiones subjetivas en un protocolo matemático homogéneo, eliminando la duplicación innecesaria de comprobaciones en bucles iterativos y garantizando que el parser mantenga un comportamiento predecible en cualquier nivel del árbol sintáctico.
2. **La superación del modelo canónico:** El hallazgo de las limitaciones del esquema clásico de Wirth ante construcciones asimétricas, particularmente en la unión entre guardianes opcionales y terminales obligatorios, la cabeza opcional de **expresion\_simple** y la necesidad de una salida temprana (Regla 9) para cortar cascadas en el origen, evidenció que un compilador robusto exige reconfiguraciones activas sobre la teoría de base.
3. **Rigor metodológico y verificación por mutación:** La adopción de un flujo ágil desacoplado (Roadmap de 39 tickets, Integración Continua estricta con 76 casos y pruebas de regresión bajo *timeout*) sacó el juicio subjetivo del camino crítico. El análisis de mutación sobre los 27 puntos de recuperación nos dio la certeza matemática de que no existe código defensivo ocioso: cada test() implementado cumple un rol indispensable y verificable en la resincronización.
4. **Comprensión de los límites de la técnica:** El proyecto nos enseñó a discernir con claridad los alcances y fronteras de la recuperación antipánica. Reconocer que los mensajes desfasados ante tokens inesperados previos a un delimitador o los falsos positivos derivados de separadores opcionales son el costo intrínseco del descarte de tokens —y que su solución definitiva pertenece a la recuperación a nivel de frase mediante inserción o reemplazo— sienta las bases conceptuales para las etapas siguientes.

En conclusión, entregamos un analizador sintáctico que no solo acepta con fidelidad el lenguaje válido y rechaza el inválido, sino que se recupera de manera localizada y limpia.

A su vez el parser queda así plenamente preparado para recibir la Tabla de Símbolos y la infraestructura semántica en la Entrega 2.